

课程须知

- 本课程适用软件版本: MWORKS.Syslab2023b 1030
- 本课程示例运行需要软件首选项加载: ?
 - 基础库(TyBase)
 - 数学库(TyMath)
 - 图形库(TyPlot)
 - 符号计算库(TySymbolicMath)

MWORKS.Syslab符号数学工具箱应用

刘尚

苏州同元软控信息技术有限公司

2023-08-12



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. 工具箱功能概况

2. 符号数学基础

3. 线性代数

4. 符号微分

5. 公式运算和简化

6. 数学函数

1. 符号数学工具箱功能概述

1 符号数学基础

- 符号表达式创建
- 基本操作
- 运算符和基本运算
- 符号和数值之间的转换

2 线性代数

- 线性方程
- 矩阵运算与变换
- 矩阵结构与属性
- 特殊矩阵

3 符号微分

- 表达式导数
- 矢量分析

4 公式运算和简化

- 简化
- 展开
- 代入

5 数学函数

- 三角函数
- 特殊函数
- 三角积分、椭圆积分
- 其他特殊函数

目录

1. 工具箱功能概况

2. 符号数学基础

3. 线性代数

4. 符号微分

5. 公式运算和简化

6. 数学函数

2. 符号数学基础 — 符号表达式创建与基本操作

- 符号表达式创建函数提供创建符号变量、表达式和函数的方法
- 基本操作函数提供算术运算、关系运算、数字运算等函数功能

一. 符号表达式创建

提供创建符号表达式以及基础运算的方案

分布检验函数名	说明
@varialbes	创建符号变量、数组、表达式
arguments	获取符号表达式操作符组成
get_variables	获取表达式中变量
operation	获取表达式中操作符
+ - * /等	算术运算

2. 符号表达式创建与基本操作

二. 符号表达式创建——示例

创建符号变量

```
@variables t::Real x::Number y::Complex z(t)
```



```
4-element Vector{Any}:  
 t  
 x  
 y  
 z(t)
```

利用符号变量组成符号表达式:

```
expr = x + y + sin(z)
```



```
expr = (y) + x + sin(z(t))
```

创建符号数组

```
@variables x[1:3] y[1:5,1:5] z[1:2,1:2,1:2]
```



```
3-element  
Vector{Symbolics.Array{Num}}:  
 x[1:3]  
 y[1:5,1:5]  
 z[1:2,1:2,1:2]
```

计算符号数组维度:

```
ndims.([x,y,z])
```



```
3-element Vector{Int64}:  
 1  
 2  
 3
```

创建符号矩阵

```
@variables x
```

```
M = [x x^3; x^2 x^4]
```



```
M = 2×2 Matrix{Num}:  
 x      x^3  
 x^2    x^4
```

计算矩阵行列式:

```
det(M)
```



```
ans = 0
```


目录

1. 工具箱功能概况

2. 符号数学基础

3. 线性代数

4. 符号微分

5. 公式运算和简化

6. 数学函数

3. 符号数学线性代数

一. 符号数学线性代数介绍

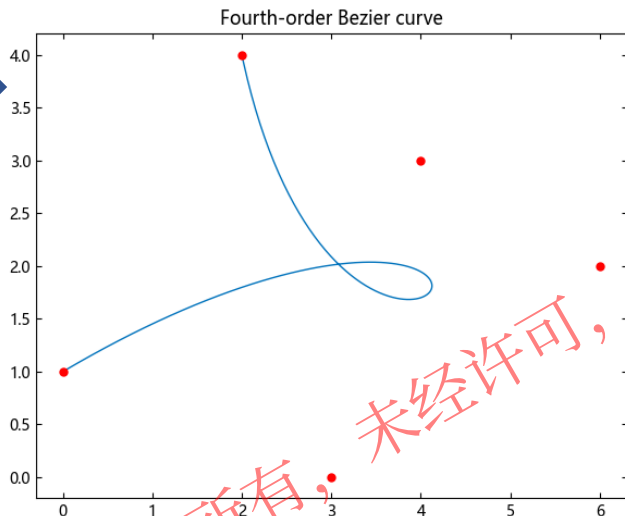
- **矩阵运算与变换**: 符号数学线性代数可以进行矩阵的加法、乘法、转置、逆等运算。它可以对矩阵进行线性变换, 包括平移、旋转、缩放和投影等。通过符号运算和代数表达式的推导, 可以得到精确的运算结果和变换关系。
- **线性方程**: 符号数学线性代数可以求解线性方程组的解析解。通过符号运算和代数表达式的推导, 可以得到方程组的通解或特解。它可以用于解决线性方程组的求解问题, 包括齐次方程组和非齐次方程组。
- **矩阵因式化和分解**: 符号数学线性代数可以对矩阵进行分解, 以获得矩阵的简化形式或特定结构的分解形式。例如, 可以对矩阵进行LU分解、QR分解等。
- **标准形式与特殊矩阵**: 符号数学线性代数可以将矩阵转化为标准形式或特殊矩阵。例如, 可以将矩阵转化为对角矩阵、上三角矩阵、Jordan标准形或Hessenberg矩阵等。这些转化可以通过符号运算和代数表达式的操作来实现, 揭示矩阵的结构和性质。
- **矩阵函数**: 符号数学线性代数可以处理矩阵函数的计算和求解。它可以通过符号运算和代数表达式的操作, 对矩阵进行函数运算, 例如指数函数、对数函数、三角函数和多项式函数等。这样可以对矩阵进行复杂运算和变换。

3. 符号数学线性代数

二. 符号数学线性代数示例——伯恩斯坦矩阵函数

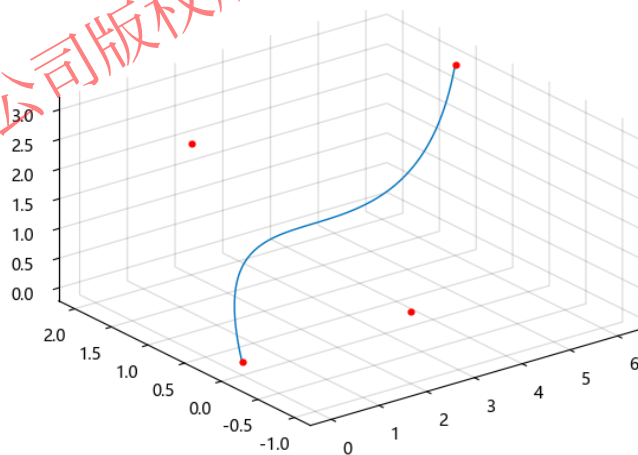
示例一：构建并绘制二维贝塞尔曲线

```
# 创建一个矩阵，每一行代表一个控制点
P = [0 1; 4 3; 6 2; 3 0; 2 4]
# 计算四阶伯恩斯坦矩阵B:
@variables t
B = bernsteinMatrix(4, t)
# 构建贝塞尔曲线:
bezierCurve = Symbolics.simplify.(B*P)
# 绘制曲线并添加控制点:
xt = build_function(bezierCurve[1],t)
yt = build_function(bezierCurve[2],t)
f_xt = eval(xt)
f_yt = eval(yt)
fplot(f_xt, f_yt, [0, 1])
hold("on")
scatter(P[:, 1], P[:, 2]; filled=true,color="r")
title("Fourth-order Bezier curve")
hold("off")
```



示例二：构建并绘制三维贝塞尔曲线

```
# 构建由 4 x 3 的控制点矩阵P指定的三阶贝塞尔曲线。
P = [0 0 0; 2 2 2; 2 -1 1; 6 1 3]
# 计算三阶伯恩斯坦矩阵B:
@variables t
B = bernsteinMatrix(3, t)
# 构建贝塞尔曲线:
bezierCurve = Symbolics.simplify.(B*P)
# 绘制曲线并添加控制点:
xt = build_function(bezierCurve[1],t)
yt = build_function(bezierCurve[2],t)
zt = build_function(bezierCurve[3],t)
f_xt = eval(xt)
f_yt = eval(yt)
f_zt = eval(zt)
fplot3(f_xt, f_yt, f_zt, [0, 1])
hold("on")
scatter3(P[:, 1], P[:, 2], P[:, 3];
filled=true,color="r")
hold("off")
```



备注:本页示例需要Syslab2023b软件支持

目录

1. 工具箱功能概况

2. 符号数学基础

3. 线性代数

4. 符号微分

5. 公式运算和简化

6. 数学函数

4. 符号微分

一. 符号微分定义及基本规则

符号微分的定义：

给定一个函数 $f(x)$ ，其中 f 是关于变量 x 的函数，其符号微分 df/dx 表示函数 f 在给定点处关于变量 x 的导数。符号微分的结果是一个符号表达式，它表示了函数 f 在每个点处的导数。

符号微分的基本规则：

- **线性规则：**微分是线性的，即对于任意的常数 c ，函数 f 和 g ，有 $d(c*f + g)/dx = c * df/dx + dg/dx$ 。
- **幂规则：**对于函数 $f(x) = x^n$ ，其中 n 是实数常数，有 $d(x^n)/dx = n * x^{(n-1)}$ 。
- **乘积规则：**对于函数 $f(x) = u(x) * v(x)$ ，有 $d(uv)/dx = u * dv/dx + v * du/dx$ ，其中 u 和 v 是关于 x 的函数。
- **链式法则：**对于复合函数 $f(g(x))$ ，有 $d(f(g(x)))/dx = (df/dg) * (dg/dx)$ ，其中 g 是关于 x 的函数， f 是关于 g 的函数。

Tips: 符号微分的定义是基于代数和符号计算的概念，它涉及对符号表达式进行微分操作，而不涉及具体的数值计算。

4. 符号微分

二. 符号微分——矢量分析示例

示例1：求向量场的散度

求向量场 $V(x, y, z) = (x, 2y^2, 3z^2)$ 关于向量 $X = (x, y, z)$ 的散度

```
@variables x y z
field = [x 2*y^2 3*z^3];
vars = [x y z];
TySymbolicMath.divergence(field, vars)
```

➡ $\text{ans} = 1 + 4y + 9(z^2)$

求这个标量函数梯度的散度。结果是标量函数的拉普拉斯算子。

```
@variables x y z
f = x^2 + y^2 + z^2;
TySymbolicMath.divergence(TySymbolicMath.gradient(f, vars), vars)
```

➡ $\text{ans} = 6$

示例2：从电场中求电荷密度

微分形式的高斯定律表明，电场的发散度与电荷密度成正比。

$$\vec{\nabla} \cdot \vec{E}(\vec{r}) = \frac{\rho(\vec{r})}{\epsilon_0}$$

求电场的电荷密度 $\vec{E} = x^2\hat{i} + y^2\hat{j}$

```
@variables x y ep0
E = [x^2 y^2];
rho = TySymbolicMath.divergence(E, [x y])*ep0
```

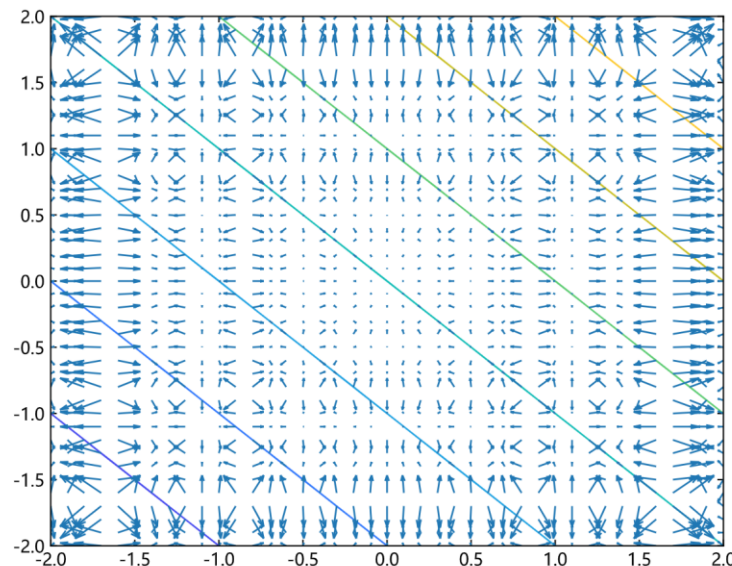
➡ $\text{rho} = \text{ep0}*(2x + 2y)$

将 $\text{ep0} = 1$ 的 $-2 < x < 2$ 和 $-2 < y < 2$ 的电场和电荷密度可视化。使用 `meshgrid2` 创建 x 和 y 值的网格。通过使用 `substitute` 替换网格值来查找电场和电荷密度的值。通过使用单元阵列作为 `substitute` 的输入，同时将网格值 `xPlot` 和 `yPlot` 代入电荷密度 `rho`。

```
rho = substitute(rho, Dict{ep0=>1})
v = -2:0.1:2;
xPlot, yPlot = meshgrid2(v);
Ex = substitute(E[1], Dict{x=>xPlot})
Ey = substitute(E[2], Dict{y=>yPlot})
rhoPlot = substitute(rho, Dict([x=>xPlot, y=>yPlot]))
```

使用 `quiver` 绘制电场。使用等高线叠加电荷密度。等高线表示电荷密度的值。

```
contour(value.(xPlot), value.(yPlot), value.(scalarize(rhoPlot)))
hold("on")
quiver(xPlot, yPlot, value.(scalarize(Ex)), value.(scalarize(Ey)), length=2.5e13)
hold("off")
```



目录

1. 工具箱功能概况

2. 符号数学基础

3. 线性代数

4. 符号微分

5. 公式运算和简化

6. 数学函数

5. 公式运算和简化

符号数学工具箱提供简化或修改表达式，替换部分表达式的函数。现已支持的函数列表如下：

公式重排和重写

函数名	说明
arguments	获取 x 的参数
expand	在加法上分布乘法来扩展表达式
factor	因式分解
div	多项式除法的商
lhs	方程的左侧
rhs	方程的右侧
substitute	符号替换

公式简化

函数名	说明
simplify	代数简化
simplify_fractions	化简符号有理表达式
flatten_fractions	展平相加的嵌套分式
substitute_expr!	符号替换
quick_cancel	代数简化



5. 公式运算和简化

一. 公式重排和重写

示例一：使用 `arguments` 函数返回表达式子项

语法: `arguments(x)`

定义符号变量

```
@variables t x y z(t)
```

```
ans = 4-element Vector{Num}:  
 t  
 x  
 y  
 z(t)
```

将num型解封装

```
expr = value(x + y + sin(z))
```

```
expr = x + y + sin(z(t))
```

返回表达式子项

```
arguments(expr)
```

```
ans = 3-element Vector{Symbolic{Real}}:  
 x  
 y  
 sin(z(t))
```

示例二：使用 `substitute` 函数进行符号替换

语法: `substitute(expr, dict; fold=true)`

在表达式中用 4 替换 a

```
@variables a b
```

```
substitute(a + b, Dict{a=> 4})
```

```
ans = 4 + b
```

不计算可计算的表达式

```
@variables a b
```

```
substitute(sin(a)*cos(b), Dict([a=>1, b=>2]), fold=false)
```

```
ans = sin(1)*cos(2)
```

用 3×3 幻方矩阵替换此表达式中的变量 a。 请注意，常量 1 扩展为 3×3 矩阵，其所有元素都等于 1。

```
b = magic(3)
```

```
@variables a[1:3,1:3] t
```

```
a = scalarize(a)
```

```
a1 = exp.(t.*a).+1
```

```
substitute(a1, Dict(a.=>b))
```

```
ans = 3×3 Matrix{Num}:
```

```
 1 + exp(8t)  1 + exp(t)  1 + exp(6t)  
 1 + exp(3t)  1 + exp(5t)  1 + exp(7t)  
 1 + exp(4t)  1 + exp(9t)  1 + exp(2t)
```

5. 公式运算和简化

二. 公式简化

示例：使用 `simplify` 函数进行代数简化

语法：`simplify(x;expand=false,threaded=false, thread_subtree_cutoff=100, rewriter=nothing))`

简化表达式

```
@variables x a b c
```

```
S = Symbolics.simplify(sin(x)^2 + cos(x)^2)
```



$S = 1$

对化简结果展开

对表达式进行直接化简，不会对括号进行直接展开

```
@variables x
```

```
S1 = Symbolics.simplify(x*(x+1)+1)
```



$S1 = 1 + x*(1 + x)$

使用`expand`参数展开表达式

```
S2 = Symbolics.simplify(x*(x+1)+1,expand = true)
```



$S2 = 1 + x + x^2$

备注:本页示例需要Syslab2023b软件支持

目录

1. 工具箱功能概况

2. 符号数学基础

3. 线性代数

4. 符号微分

5. 公式运算和简化

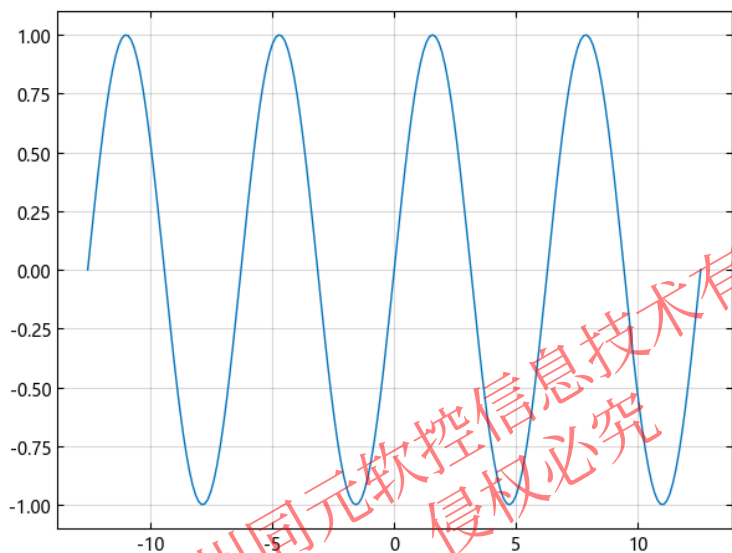
6. 数学函数

6. 数学函数

一. 三角函数

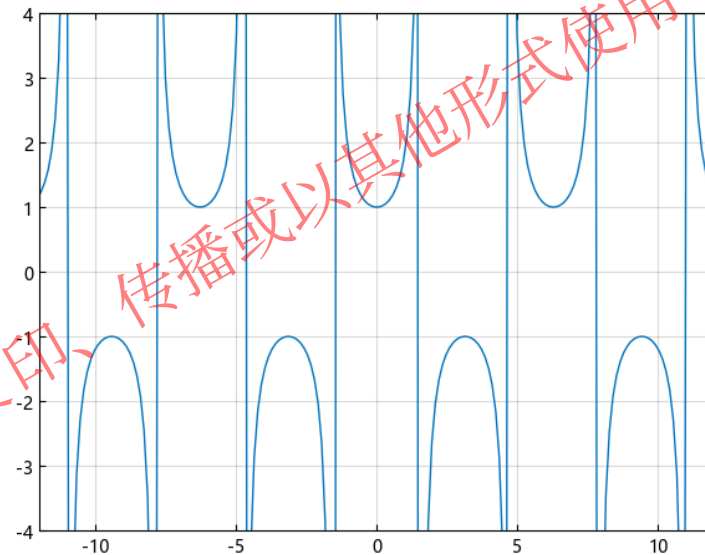
示例一：在域 $-4\pi \leq x \leq 4\pi$ 上绘制正弦函数图

```
@variables x
sinf = x-> sin(x)
fplot(sinf,[-4*pi 4*pi])
grid("on")
```



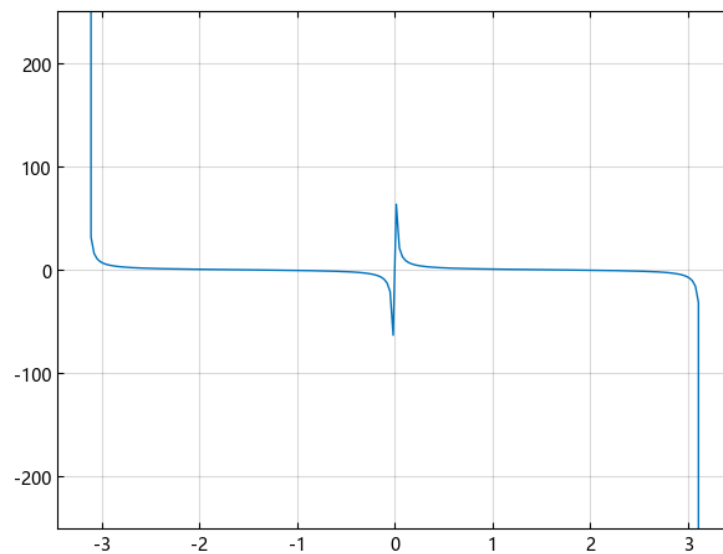
示例二：在域 $-4\pi \leq x \leq 4\pi$ 上绘制正割函数图

```
@variables x
secf = x-> sec(x)
fplot(secf,[-4*pi 4*pi])
xlim([-12 12])
ylim([-4 4])
grid("on")
```



示例三：在域 $-\pi \leq x \leq \pi$ 上绘制余切函数图

```
@variables x
cotf = x-> cot(x)
fplot(cotf,[-pi pi])
ylim([-250 250])
grid("on")
```



6. 数学函数

二. 其他特殊函数

示例一： zeta — 黎曼 zeta 函数

语法: `zeta(z)`

`zeta(n,z)`

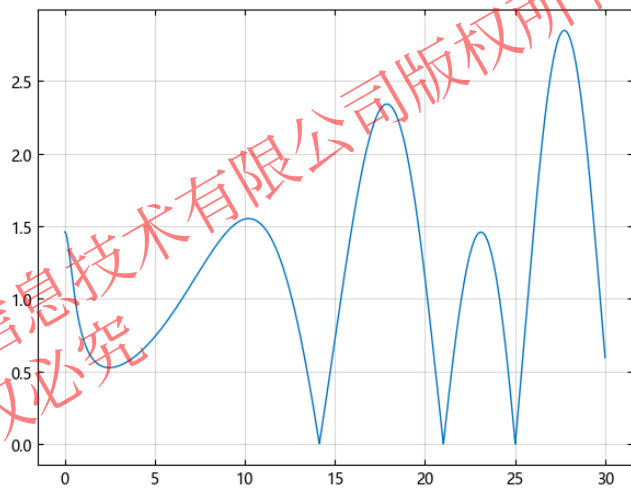
计算黎曼zeta函数

```
zeta.([0.7 im 4 11/3]) ➡ ans = 1x4 Matrix{ComplexF64}:  
-2.77839+0.0im    0.00330022-0.418155im  
1.08232+0.0im    1.10941+0.0im
```

绘制黎曼zeta函数

对于 `zeta(x+iy)` 可以在 $x = 1/2$ 处得到黎曼zeta函数零点, 在 $0 < y < 30$ 绘制函数绝对值函数

```
x = 1/2  
y = 0:0.01:30  
plot(y,abs.(zeta.(x.+im*y)))  
grid("on")
```



示例二： lambertw — 朗博w函数 (又称欧米茄函数)

语法: `P = lambertw(x)` `P = lambertw(x,k)`

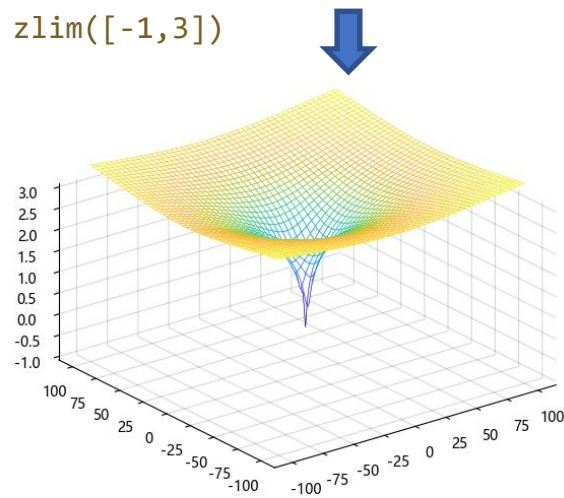
计算lambertw函数

```
A = [0 -1/exp(1); pi im] ➡ ans = 2x2 Matrix{ComplexF64}:  
0.0+0.0im    -1.0+0.0im  
TySymbolicMath.lambertw.(A) 1.07366+0.0im 0.374699+0.576413im
```

复平面上的 Lambert W 函数图

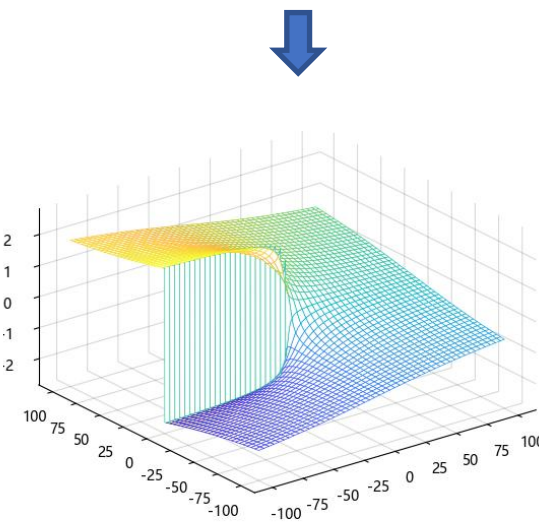
绘制lambertw实部值

```
X,Y = meshgrid2(-100:0.1:100)  
Z = TySymbolicMath.lambertw.(X+im*Y)  
mesh(X,Y,real.(Z))  
zlim([-1,3])
```



绘制lambertw虚部值

```
mesh(X,Y,imag.(Z))
```



建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听